

2.1 [5] <\$2.2> For the following C statement, what is the corresponding MIPS assembly code? Assume that the variables *f*, *g*, *h*, and *i* are given and could be considered 32-bit integers as declared in a C program. Use a minimal number of MIPS assembly instructions.

```
f = g + (h - 5);
```

Assuming *f*, *g*, and *h* are stored in \$s0-\$s3

```
addi $t0, $s2, -5
```

```
add $s0, $s1, $t0
```

2.2 [5] <\$2.2> For the following MIPS assembly instructions above, what is a corresponding C statement?

```
add f, g, h
```

```
add f, i, f
```

```
f = g + h
```

```
f += i
```

2.4 [5] <\$\$2.2, 2.3> For the MIPS assembly instructions below, what is the corresponding C statement? Assume that the variables *f*, *g*, *h*, *i*, and *j* are assigned to registers \$s0, \$s1, \$s2, \$s3, and \$s4, respectively. Assume that the base address of the arrays *A* and *B* are in registers \$s6 and \$s7, respectively.

```
sll $t0, $s0, 2      # $t0 = f * 4
```

```
add $t0, $s6, $t0     # $t0 = &A[f]
```

```
sll $t1, $s1, 2      # $t1 = g * 4
```

```
add $t1, $s7, $t1     # $t1 = &B[g]
```

```
lw $s0, 0($t0)        # f = A[f]
```

```
addi $t2, $t0, 4
```

```
lw $t0, 0($t2)
```

```
add $t0, $t0, $s0
```

```
sw $t0, 0($t1)
```

```
B[g] = A[f] + A[f + 1]
```

2.12 Assume that registers \$s0 and \$s1 hold the values 0x80000000 and 0xD0000000, respectively.

2.12.1 [5] <\$2.4> What is the value of \$t0 for the following assembly code?

```
add $t0, $s0, $s1
```

$$\begin{array}{r} 0x80000000 \\ + 0xD0000000 \\ \hline \end{array}$$

0x150000000

2.12.2 [5] <\$2.4> Is the result in \$t0 the desired result, or has there been overflow?

There has been overflow.

2.12.3 [5] <\$2.4> For the contents of registers \$s0 and \$s1 as specified above, what is the value of \$t0 for the following assembly code?

```
sub $t0, $s0, $s1
```

$$\begin{array}{r} 0x80000000 \\ - 0xD0000000 = (+ 0x30000000) \\ \hline \end{array}$$

0xB0000000

2.12.4 [5] <\$2.4> Is the result in \$t0 the desired result, or has there been overflow?

No overflow here!

2.12.5 [5] <\$2.4> For the contents of registers \$s0 and \$s1 as specified above, what is the value of \$t0 for the following assembly code?

```
add $t0, $s0, $s1  
add $t0, $t0, $s0
```

$$\begin{array}{r} 0x50000000 \\ + 0x80000000 \\ \hline \end{array}$$

0xD0000000

2.12.6 [5] <\$2.4> Is the result in \$t0 the desired result, or has there been overflow?

No overflow!!

2.16 [5] <\$2.5> Provide the type, assembly language instruction, and binary representation of instruction described by the following MIPS fields:

op=0, rs=3, rt=2, rd=3, shamt=0, funct=34

```
sub $v1, $v1, $v0
```

Instruction:

rd=3 → \$v1 (destination)

rd=3 → \$v1 (source 1)

rt=2 → \$v0 (source 2)

Binary Representation:

op	rs	rt	rd	shamt	funct
000000	00011	00010	00011	00000	100010

2.27 [5] <\$2.7> Translate the following C code to MIPS assembly code. Use a minimum number of instructions. Assume that the values of a, b, i, and j are in registers \$s0, \$s1, \$t0, and \$t1, respectively. Also, assume that register \$s2 holds the base address of the array D.

```
for(i=0; i<a; i++)
    for(j=0; j<b; j++)
        D[4*j] = i + j;
```

```
addi $t0, $zero, 0
```

```
outer:
```

```
    slt $t3, $t0, $s0
```

```
    beq $t3, $zero, done
```

```
    addi $t1, $zero, 0
```

```
inner:
```

```
    slt $t3, $t1, $s1
```

```
    beq $t3, $zero, outer_inc
```

```
    add $t3, $t0, $t1
```

```
    sll $t4, $t1, 4
```

```
    add $t4, $s2, $t4
```

```
    sw $t3, 0($t4)
```

```
    addi $t1, $t1, 1
```

```
    j inner
```

```
outer_inc:
```

```
    addi $t0, $t0, 1
```

```
    j outer
```

```
done:
```